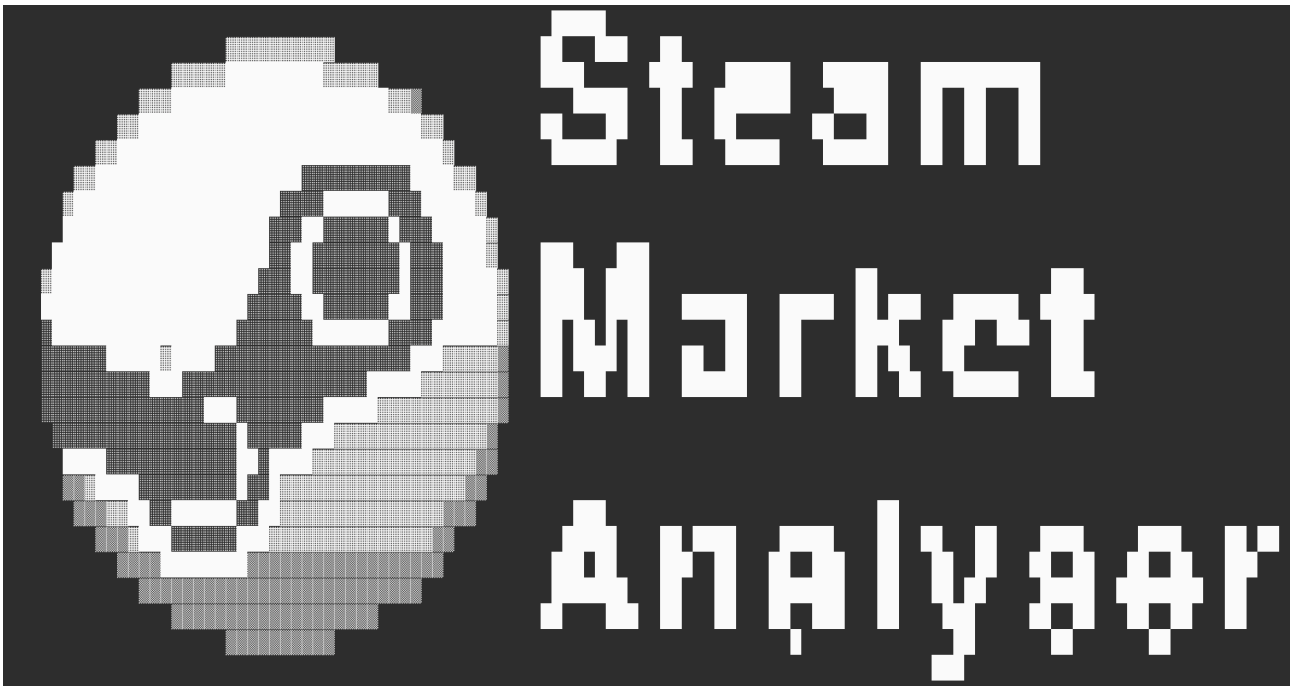


Steam Market Analyser

Introduction

The Steam Market Analyser (SMA) takes the Steam Games Dataset (Bustos Roman, 2022) from Kaggle and aims to provide useful display and analysis of this data including through the application of filters and statistical analysis.

This analysis of data is provided through a suite of analysis and transformation tools that allow the user to perform a number of operations on the dataset.

Word count (minus appendices, references, and figures): 1492

Justified Program Design

Program Scope

As outlined in the introduction there are a few key things the program must do. The first was having a clean and readable means of displaying the data. The use of a library such as Rich (Textualize, 2026) makes this possible as it allows for the display of neat tables, and allowing the user to specify what columns to display along with pagination and filtering further helps to break down the large dataset.

Then to provide function, and to meet the assessment criteria, the program must count the number of records, and perform operations on the dataset such as transformation and statistical analysis. Both of these are done with the use of the additional libraries Pandas (Pandas Developers, 2026) and NumPy (Numpy Developers, 2026). Both the display of the data, data transformation, and analysis require user input and error handling.

Something I took to heart from the assessment criteria page was the fact this program must run unattended and in an unknown environment. Thus, library availability is unknown so I want to ensure some form of the program will run no matter what. To this end I wanted to have standard library fallbacks for the two most critical libraries; those being Rich and Pandas.

Rather than hard code the analysis and transformations, I want the user to be able to pick what analysis is performed on the various areas of the dataset. Whilst this presents more work programmatically, it allows for the most flexibility - especially when dealing with the 50 or so columns the dataset provides.

Finally, the program should be able to export the computed data, again with the user able to pick the format - raw data as a csv or graphing using MatPlotLib (MatPlotLib Developers, 2026).

Program Design

Initially I was set on a small program with hard coded operations and little defined structure. However, after some thought I opted to refactor and implement a tried and tested model, view, controller (MVC) architecture which Laravel (2025) describe as a way to keep code clean and sanity intact. I'm inclined to agree with their assessment, as coming from a background of non-professional development in PHP and Angular MVC is always how I try to layout projects.

Models	Store the dataset and hold our applications state. Perform manipulation on the dataset in response to actions requested by the services.
Views	Responsible for outputting the constructed UI from the controllers
Controllers	Call upon logic functions and the views, handle user input. Orchestrates interaction between the user, the view, and the models.
Services	Large complex functions that may or may not directly relate to controllers.
Utilities	Small helper functions used throughout the application
Figure 1: Model view controller explanation	

This layout helps keep concerns separated, limiting the effects modifying one part of the code has on the other pieces. The drawback is the addition of slightly more repetition, as rather than code just calling the functions it requires, application must flow from controller

to controller, which in turn call the required views and services which call the required models.

Broadly speaking the application can be divided into dataset viewing, dataset filtering, dataset transformation and analysis, dataset graphing, and dataset export. Each area utilises a controller, a view, and a service, with the operations being handled by the dataset model.

The heavy lifting of data analysis and manipulation is performed in the dataset models. At application start the feature service determines whether Pandas is available and from there the polymorphic dataset model will implement a Pandas or Nolib (no library) dataset class as appropriate. I aimed to build the dataset models - be it implemented as lists or a Pandas dataframe - in a way similar to using a model to interact with a PostgreSQL database, with clear functions to select a column's cells, calculate a logarithm on a column, or calculate an average.

Data Design

Variable, function, and class naming follows the Python Enhancement Proposals (PEP) style guide by van Rossum, Warsaw and Coghlan (2001) with functions and variables named in snake_case, and classes being named using PascalCase. In the same vein comments (notably DocString comments on functions) follow this style guide too.

The dataset is available as a csv or json and I opted to use the csv file. From there the csv will either be stored as a Pandas dataframe (if Pandas is available) or as two lists (Columns[str] containing column names, and Rows[list[str|int|float…]).

To allow for the dataset to be used throughout the application, as well as for active filters to be passed, I implemented an application state model (AppState). The content of this can be seen below with description of the various attributes. Consolidating everything into a state object is much cleaner than using global variables and goes a long way to making the code more structured.

Algorithms and Program Flow

As above, the program follows a model, view, controller architecture, with the controller loading the view and interacting with services. Data is passed through the app using the AppState class. The application holds and awaits user inputs using while loops. Throughout a variety of for loops are used to iterate over the rows in the dataset.

Design Choices & Issues

I wanted to put a big emphasis on reliability, error handling, and functioning no matter what libraries were available, so the inclusion of a feature checking and feature flag system was one of the first things implemented. This then went hand in hand with the polymorphic Pandas or Nolib Dataset implementation which I added not just for the “run anywhere” objective, but to further demonstrate programming ability - manipulating a Pandas dataset is much easier than doing so without.

The same logic can be applied to the decision to split the user interface into Rich and NoLib views, with NoLib versions being used in the event Rich wasn’t available. While the MVC structure made implementation simple in theory, due to time constraints I was unable to write new views using the Python standard library before the submission deadline.

Standardising user input using the expect_user_input() function in the user_input_handler.py utility was one of the first things I wrote and it definitely helped streamline the large amount of user input required. Having user input type checking and constraints handled in one place makes the code much easier to read.

Of course this emphasis on standard library fallbacks, user customisation, and application flexibility, made development a much longer process than intended. I find whenever I develop and program feature creep and ballooning scope is a regular occurrence, with the constant thought of “what more can I add?”

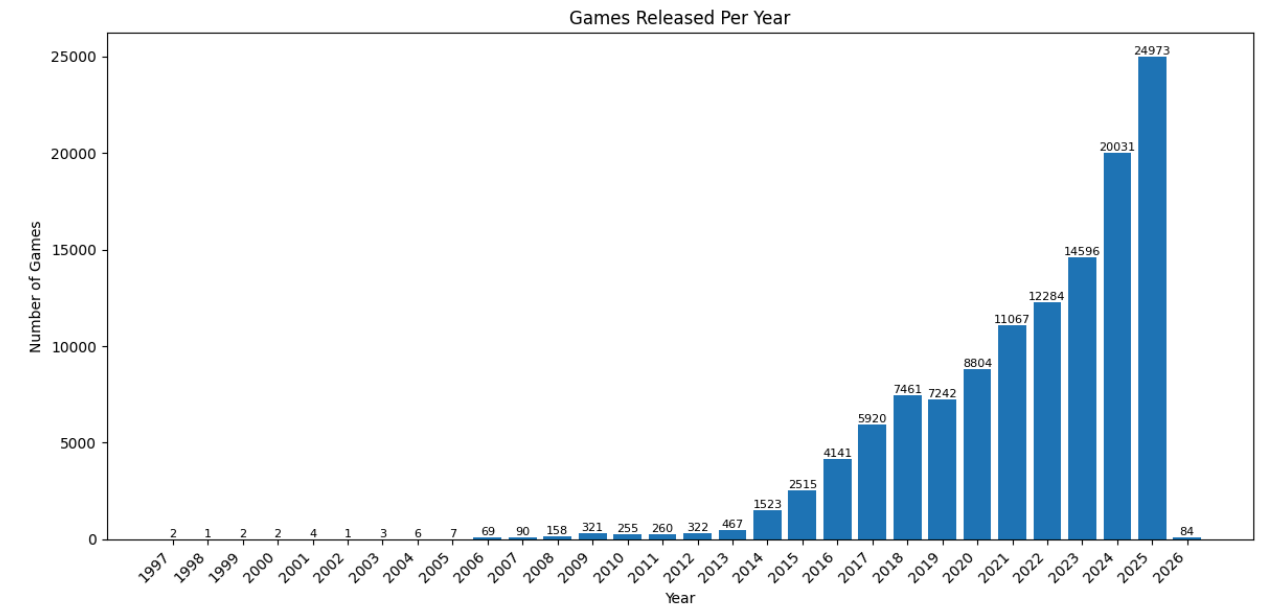
Testing

I had intended to implement the Pytest library (Pytest, 2026) and work in tests as I developed, however this was neglected and instead I relied on running the application and handling any inconsistencies or errors as I went. Once I’d finished development I created a simple test to output a line, bar, and pie chart. This test ran through the functions that would be executed if a user was to work through the menus, outputting a graph of each type: bar, line, and pie.

This test process did show an interesting error whereby I learned of duplicate games in the dataset, with Tom Clany’s Rainbow Six Siege showing 5 times. On examination of the dataset I learned it was indeed duplicated, each duplicate having a different AppID but being the same in every other respect. In response to this I added a method to remove duplicates from the dataset based on name during importation.

Results

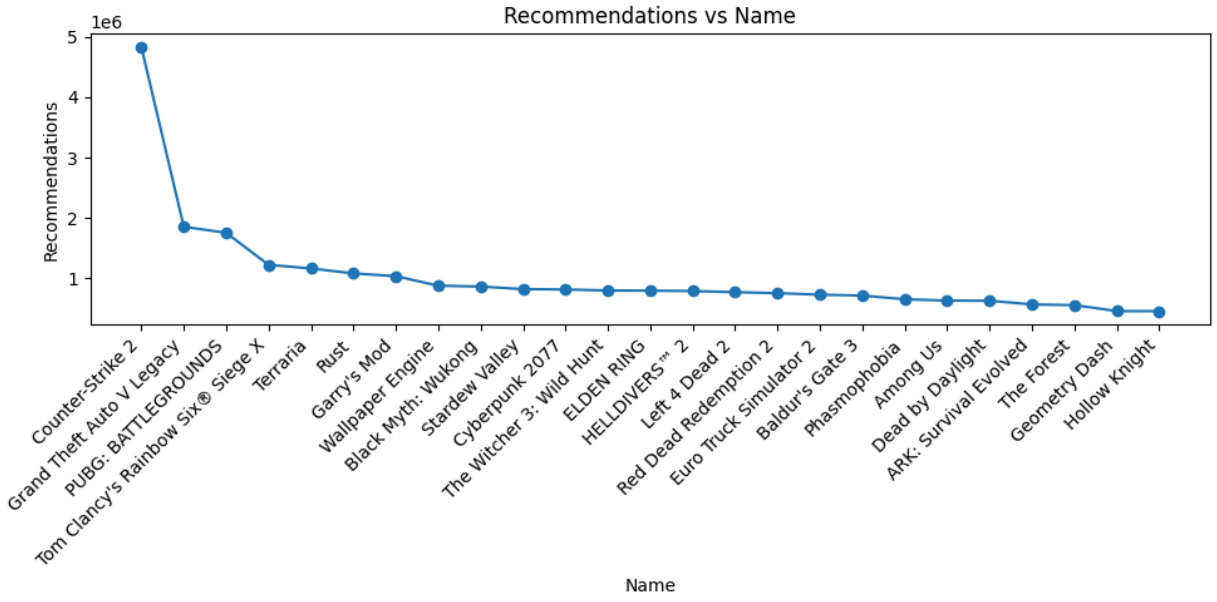
Any number of analyses can be run on the dataset, but I'll demonstrate a couple of key ones here alongside how to generate them within the application.



Methodology: "Extract Year Column", selecting "Release Date" and creating "Release Year". Then performing a "String List Value Ranking", "Release Year" as the target column, and "," as the separator, and "30" for the count. From this a bar graph can be generated selecting to create the graph from last analysis, with year as the x axis and

This shows us that the number of games released on Steam is growing year on year with dramatic growth in 2024 and 2025.

Figure 2: Games Released Per Year Graph from Steam Games Dataset generated with the Steam Market Analyser



Methodology: Select only the “Name” and “Recommendations” columns. Then in transformations and analysis select “Top N Rows by Numeric Value” choosing “Recommendations” as the rank, and the number as 25. Then generate a line graph from last analysis.

The disparity between Counter-Strike 2 (CS2) and Grand Theft Auto V’s number of recommendations is huge, which warranted further investigation. What the dataset doesn’t show, and what Steam doesn’t clearly display, is that CS2’s recommendations factor in the previous version’s (Counter-Strike: Global Offensive) recommendations too, artificially inflating the number of recommendations it has had. As CS2 is made by Valve, the same company that own Steam, one could argue this disparity is intentional.

Figure 3: Recommendations Per Game from Steam Games Dataset generated with the Steam Market Analyser

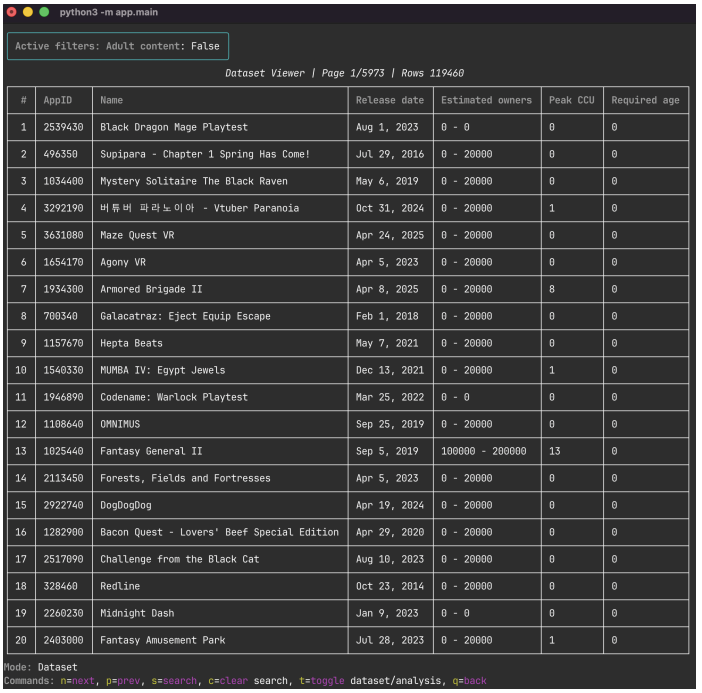
Reflection

I successfully created an application that enables the display and analysis of the Steam Games Dataset, although the program can theoretically be used with any dataset. I'm very pleased with the display of the dataset with the pagination, ability to filter, search, and select displayed columns. I'm also very happy with the decision to implement a standard library fallback for Pandas as it was a good learning experience handling tabular data without the support of a library, however the increased workload took away from developing other library fallbacks - such as a user interface without Rich.

It was also a challenge fitting in the development of a project around shift work, with many hours spent sitting in an ambulance cab with a crewmate looking on bemused as I tapped away on my laptop or stared off into space mulling over solutions.

Evaluation

The program successfully imports the Steam Games Dataset csv file into a Pandas dataframe or standard library lists. It features a comprehensive user interface which allows the user to view the dataset with filters and column selection, apply transformations, generate analyses, and export findings as raw data or as a bar, line, or pie chart. Throughout library availability, user input, transformation, and analyses are error checked and handled (gracefully where possible).



#	AppID	Name	Release date	Estimated owners	Peak CCU	Required age
1	2539430	Black Dragon Mage Playtest	Aug 1, 2023	0 - 0	0	0
2	496350	Supipara - Chapter 1 Spring Has Come!	Jul 29, 2016	0 - 20000	0	0
3	1034400	Mystery Solitaire The Black Raven	May 6, 2019	0 - 20000	0	0
4	3292190	버튜버 파라노이아 - Vtuber Paranoia	Oct 31, 2024	0 - 20000	1	0
5	3631080	Maze Quest VR	Apr 24, 2025	0 - 20000	0	0
6	1654170	Agony VR	Apr 5, 2023	0 - 20000	0	0
7	1934300	Armored Brigade II	Apr 8, 2025	0 - 20000	8	0
8	700340	Galacatraz: Eject Equip Escape	Feb 1, 2018	0 - 20000	0	0
9	1157670	Hepta Beats	May 7, 2021	0 - 20000	0	0
10	1540330	MUMBA IV: Egypt Jewels	Dec 13, 2021	0 - 20000	1	0
11	1946890	Codename: Warlock Playtest	Mar 25, 2022	0 - 0	0	0
12	1108640	OMNIMUS	Sep 25, 2019	0 - 20000	0	0
13	1025440	Fantasy General II	Sep 5, 2019	100000 - 200000	13	0
14	2113450	Forests, Fields and Fortresses	Apr 5, 2023	0 - 20000	0	0
15	2922740	DogDogDog	Apr 19, 2024	0 - 20000	0	0
16	1282900	Bacon Quest - Lovers' Beef Special Edition	Apr 29, 2020	0 - 20000	0	0
17	2517090	Challenge from the Black Cat	Aug 10, 2023	0 - 20000	0	0
18	328460	Redline	Oct 23, 2014	0 - 20000	0	0
19	2260230	Midnight Dash	Jan 9, 2023	0 - 0	0	0
20	2403000	Fantasy Amusement Park	Jul 28, 2023	0 - 20000	1	0

References

- Bustos Roman, M. (2022) *Steam Games Dataset*. Kaggle. doi: 10.34740/KAGGLE/DS/2109585. Available at: https://www.kaggle.com/ds/2109585 (Accessed: 06 February 2026).
- Textualize (2026) *rich* (Version 14.3.2) [Source code]. GitHub repository. Available at: https://github.com/Textualize/rich (Accessed: 08 February 2026).
- MatPlotLib Developers (2025) *matplotlib* (Version 3.10.8) [Source code]. GitHub repository. Available at: https://github.com/matplotlib/matplotlib (Accessed: 20 February 2026).
- NumPy Developers (2026) *numpy* (Version 2.4.2) [Source code]. GitHub repository. Available at: https://github.com/numpy/numpy (Accessed: 18 February 2026).
- Pandas Developers (2026) *pandas* (Version 3.0.0) [Source code]. GitHub repository. Available at: https://github.com/pandas-dev/pandas (Accessed: 08 February 2026).
- Laravel (2025) ‘What is MVC?’. *Laravel Learn: Getting Started with Laravel*. 25 August. Available at: https://laravel.com/learn/getting-started-with-laravel/what-is-mvc (Accessed: 26 February 2026).

- van Rossum, G., Warsaw, B. and Coghlan, N. (2001) 'PEP 8 – Style Guide for Python Code'. *Python Enhancement Proposals*. Available at: https://peps.python.org/pep-0008/ (Accessed: 16 February 2026)
- pytest-dev (2026) *pytest* (Version 9.0.2) [Source code]. GitHub repository. Available at: https://github.com/pytest-dev/pytest (Accessed: 23 February 2026).

Appendicies

Appendix 1: Dataset columns with descriptions and types		
Column Name	Description	Type
AppID	Unique Steam application identifier for the game/app.	integer
Name	Display name of the game/app.	string
Release date	Official release date of the game/app.	date
Estimated owners	Estimated number of owners (often provided as a range).	string
Peak CCU	Peak concurrent users (highest recorded concurrent players).	integer
Required age	Minimum required age to access/purchase the game/app.	integer
Price	Current base price (before discount) expressed as $	integer
Discount	Amount of discount expressed as a percentage	integer
DLC count	Number of DLC items	
About the game	Game description	string
Supported languages	Languages supported	string
Full audio languages	Languages with full audio/voice acting support.	string
Reviews	Review summary / review text field (source-dependent).	string
Header image	URL to the main header image.	string
Website	Official website URL for the game/app.	string

Support url	Support page URL for the game/app.	string
Support email	Support contact email address.	string
Windows	Whether the game/app supports Windows.	boolean
Mac	Whether the game/app supports macOS.	boolean
Linux	Whether the game/app supports Linux.	boolean
Metacritic score	Metacritic critic score (if available).	integer
Metacritic url	URL to the Metacritic page (if available).	string
User score	User score rating (source-dependent, e.g., Metacritic user score).	integer
Positive	Count of positive reviews/ratings (source-dependent).	integer
Negative	Count of negative reviews/ratings (source-dependent).	integer
Score rank	Rank/label derived from score (often sparsely populated).	string
Achievements	Count of achievements	integer
Recommendations	Count of recommendations	integer
Notes	Miscellaneous notes field	string
Average playtime forever	Average total playtime per user across all time.	integer
Average playtime two weeks	Average playtime per user over the last two weeks.	integer
Median playtime forever	Median total playtime per user across all time.	integer
Median playtime two weeks	Median playtime per user over the last two weeks.	integer
Developers	Developer name(s).	string
Publishers	Publisher name(s).	string
Categories	Category list (e.g., Single-player, Co-op).	string
Genres	Genre list.	string
Tags	Tag list (often community tags).	string
Screenshots	Screenshot URLs or screenshot data (often a list).	string

Movies	Trailer/movie URLs or movie data (often a list).	string